PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula 4 - Arquitetura de Software vs Design de Software

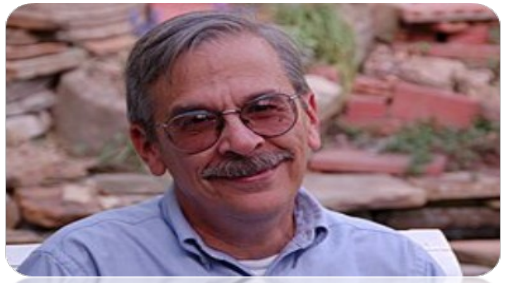
Prof.: Filipe Nhimi

Agenda

- ❑ Revisar o conceito formal de Arquitetura de Software
- ❑ Definição de Design de Software
- ❑ Diferenciar Arquitetura de Design
- ❑ Dinâmica de fixação do tema
- ❑ Conclusão

O que é Arquitetura de Software?

"Segundo Bass, Clements e Kazman, Arquitetura de Software é a **estrutura ou estruturas de um sistema**, compostas por **elementos de software**, suas propriedades visíveis externamente e as **relações** entre esses elementos."



Len Bass

Paul Clements

Rick Kazman

ESTRUTURA != CÓDIGO

ELEMENTOS != CLASSES APENAS

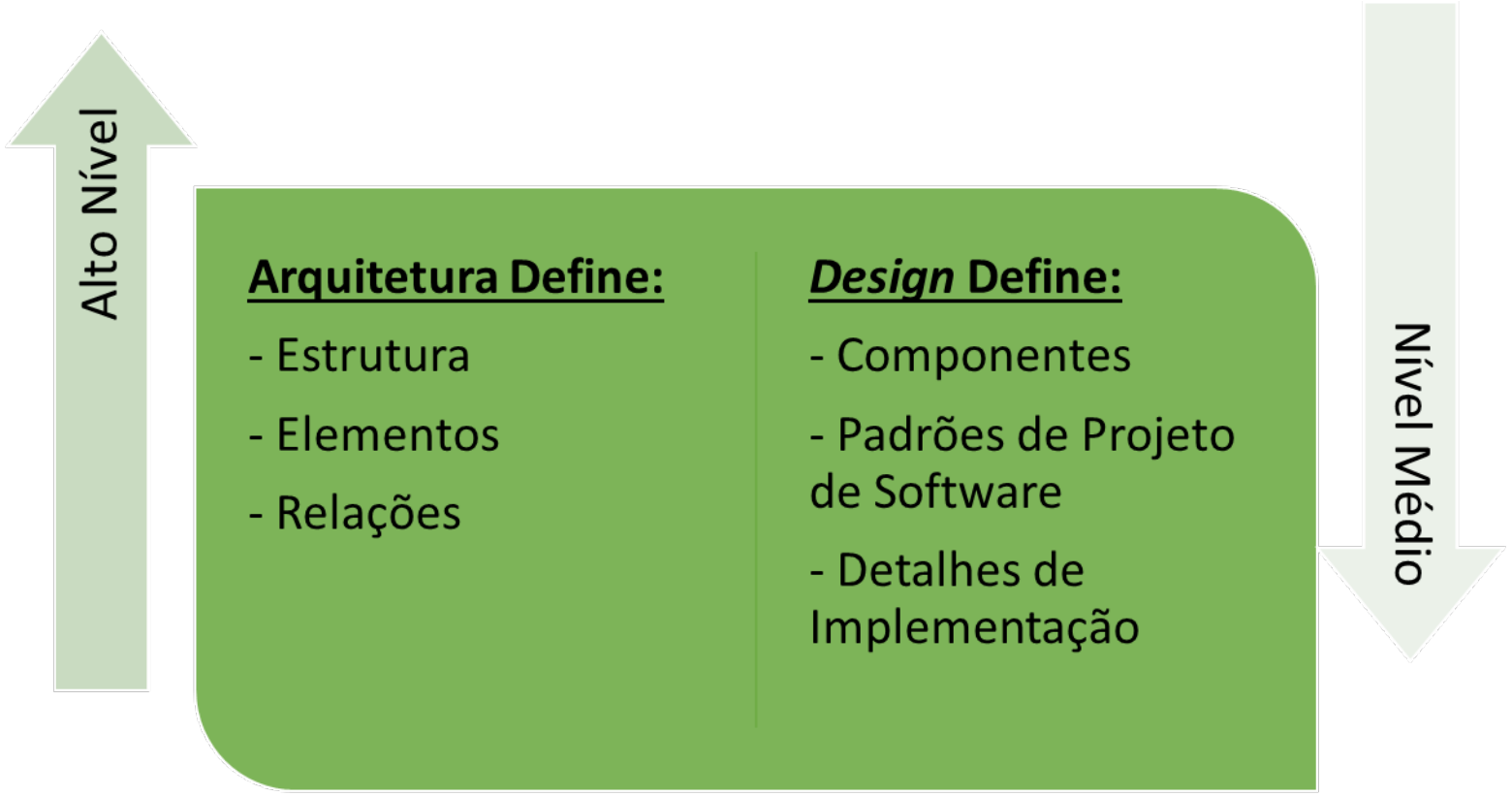
RELAÇÕES != CHAMADAS DE MÉTODO SOMENTE

O que é *Design* de Software?

"*Design* de *Software* é o processo de **definir** os **componentes internos**, suas **interfaces**, dados e **comportamento**, com o objetivo de **implementar** os **requisitos** do sistema de acordo com a arquitetura estabelecida."

*Baseada nas referências de Sommerville, Bass, Richards

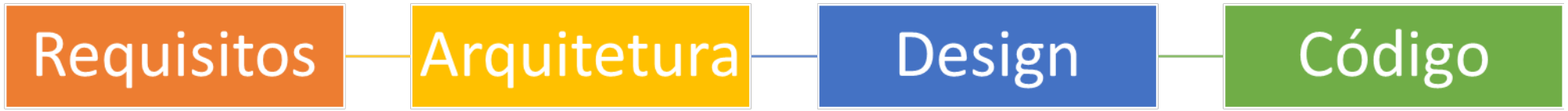
Arquitetura vs *Design* de Software



Arquitetura Responde: Como o sistema é organizado?

Design Responde: Como o sistema será implementado?

Hierarquia de Processos

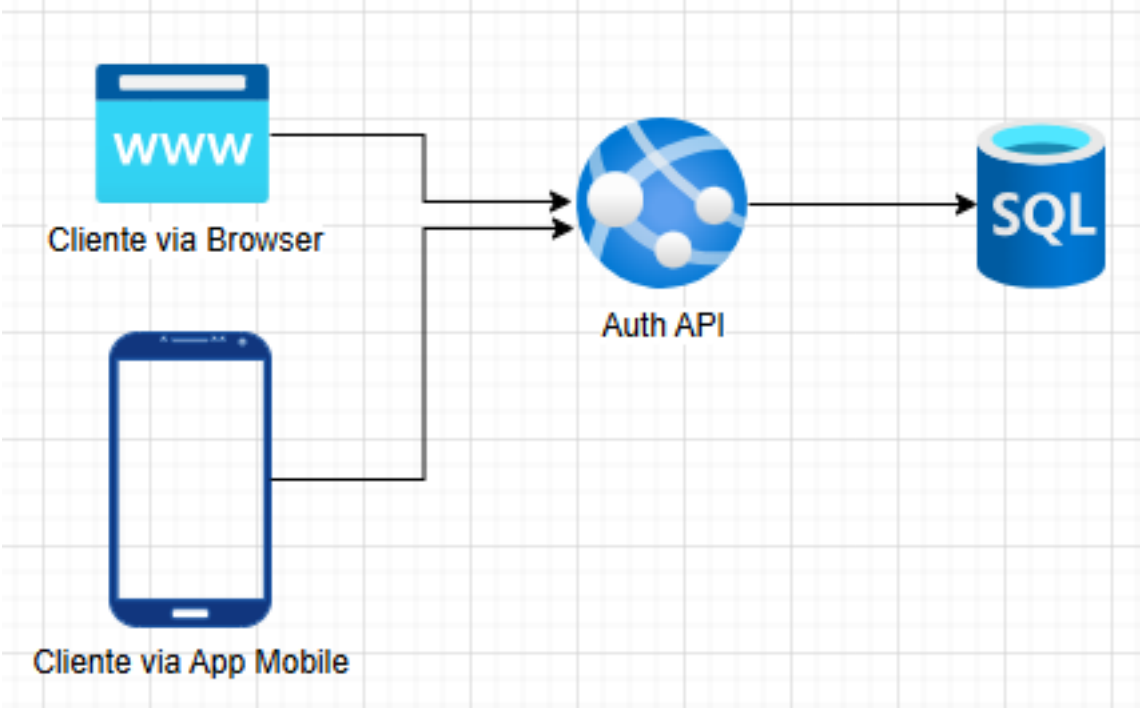


Arquitetura vs *Design* de Software

Exemplo 1: Sistema de Gestão Acadêmica (SGA)

RF-05 – Autenticar usuário do tipo “Aluno”, cadastrado na plataforma e que possui matrícula ativa

Arquitetura



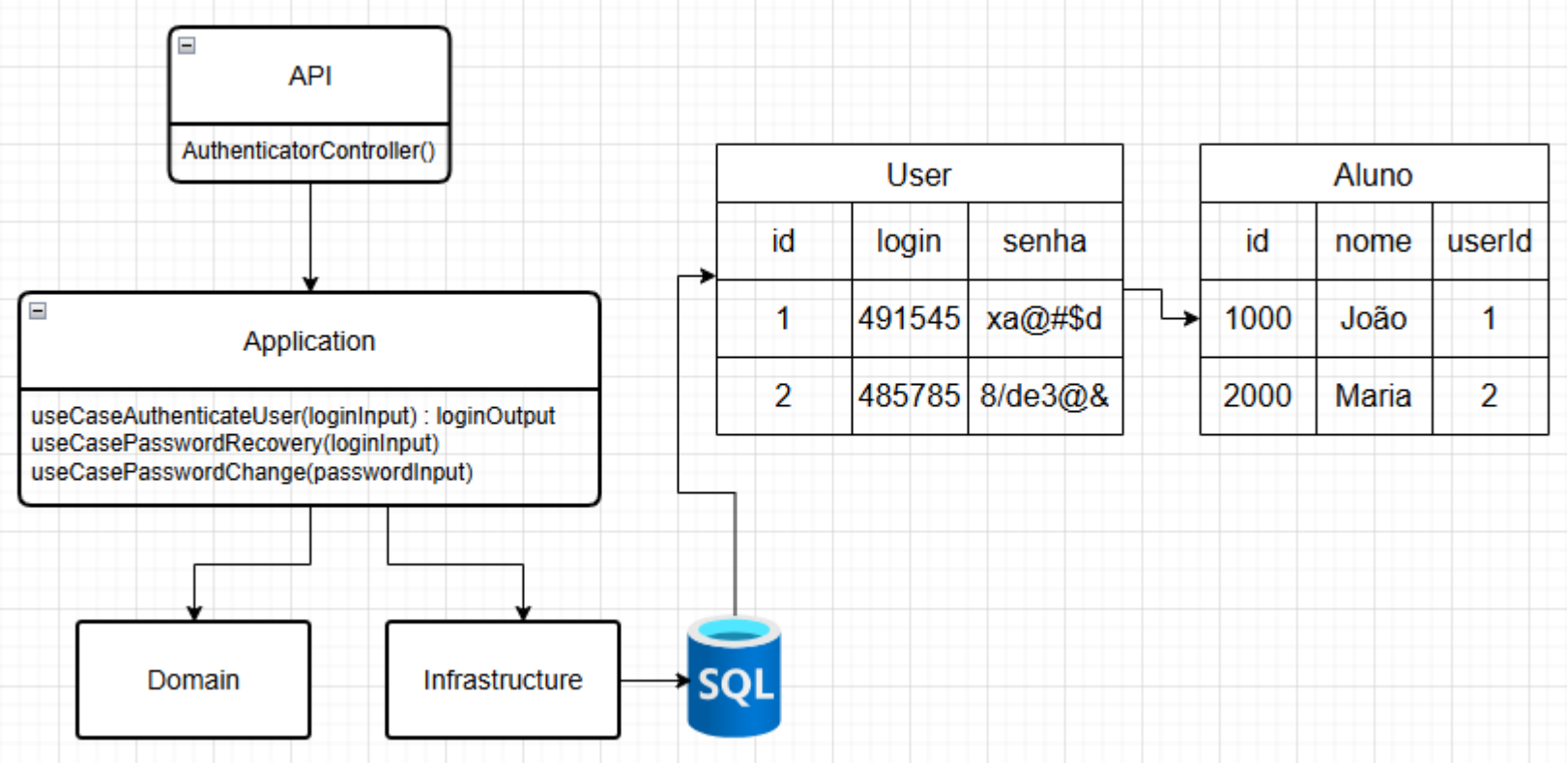
Arquitetura Responde:
Como o sistema é organizado?

Arquitetura vs *Design* de Software

Exemplo 1: Sistema de Gestão Acadêmica (SGA)

RF-05 – Autenticar usuário do tipo “Aluno”, cadastrados na plataforma e possuem matrícula ativa

Design de Software



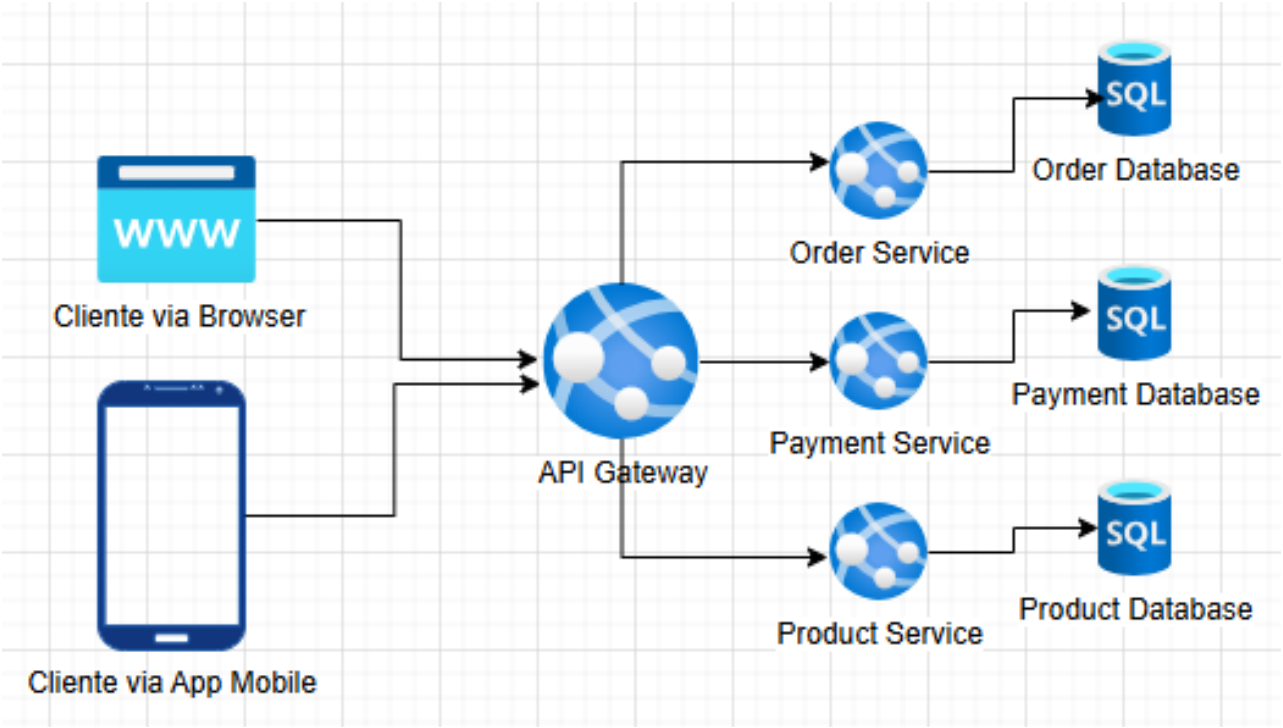
Design Responde: Como o sistema será implementado?

Arquitetura vs *Design* de Software

Exemplo 2: Sistema de *e-commerce* de loja de sapatos

RF-02 – Processar pedidos: O sistema deve permitir que um cliente finalize um pedido e registre os itens e o valor total.

Arquitetura

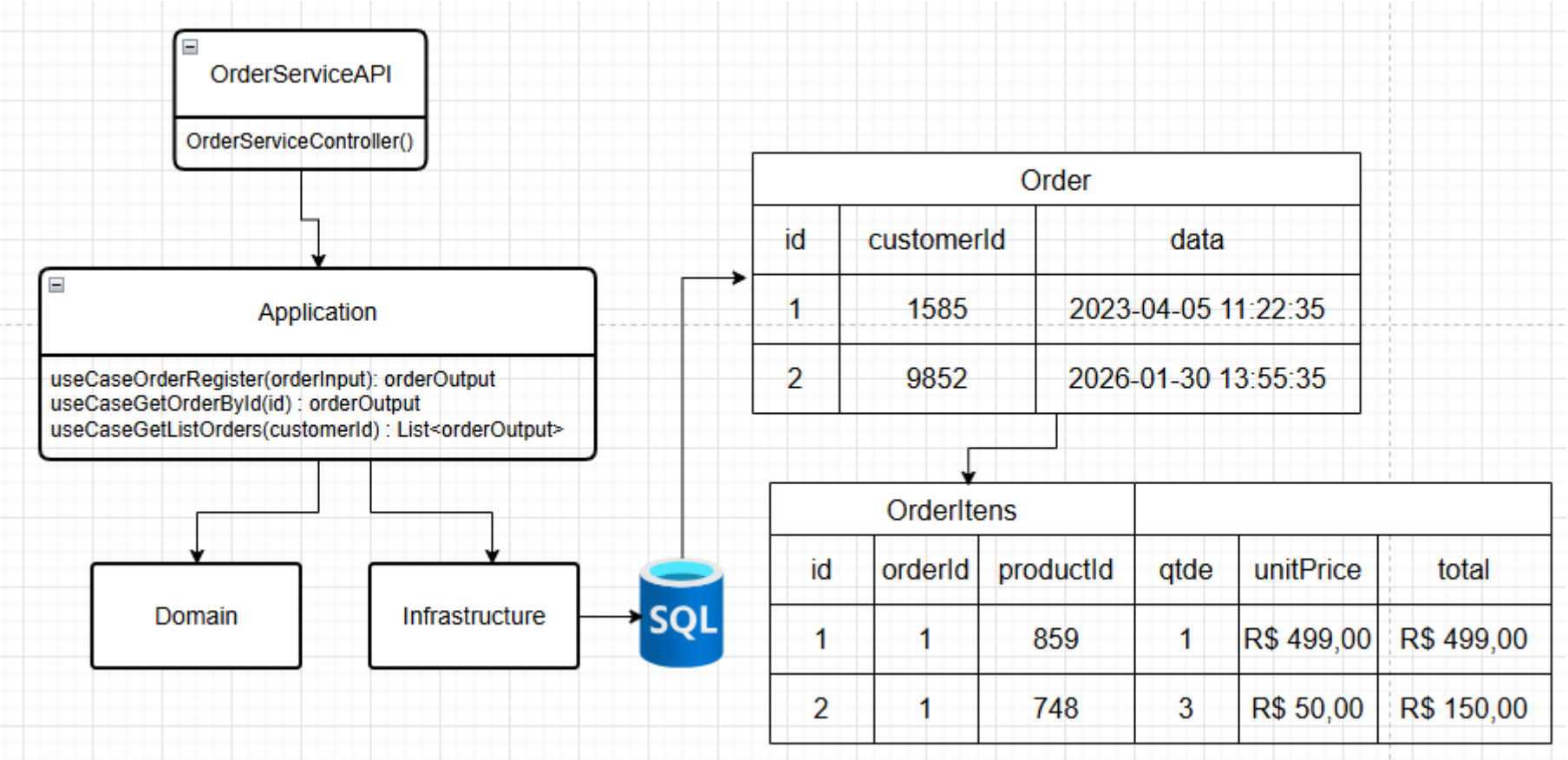


Arquitetura vs *Design* de Software

Exemplo 2: Sistema de *e-commerce* de loja de sapatos

RF-02 – Processar pedidos: O sistema deve permitir que um cliente finalize um pedido e registre os itens e o valor total.

Design de Software



Arquitetura de Software e Design de Software se materializam após uma série de decisões difíceis

Ainda que **Arquitetura de Software** e **Design de Software** sejam coisas distintas e que propósito específico, ambas tem algo em comum e que irá determinar o sucesso ou fracasso do projeto!

"É um conjunto de **decisões** difíceis de serem mudadas no futuro."
Martin Fowler

Dinâmica sobre **A**rquitetura de Software vs **D**esign de Software.
Classifique os itens com **A** ou **D**

- Sistema em microserviços ☐
- Uso de REST ☐
- Classe UsuarioService ☐
- Endpoint /api/login ☐
- Uso de mensageria ☐
- Sistema distribuído ☐
- Separação ☐
- frontend/backend ☐
- Método validar() ☐
- Classe MessageHandler ☐
- Uso de cache Redis ☐
- Classe CacheManager ☐
- API Gateway ☐
- Sistema tolerante a falhas ☐
- Camada de Repositório ☐
- Interface Repository ☐
- Comunicação assíncrona ☐
- Padrão de Projeto Adapter ☐
- Uso de PostgreSQL ☐
- Algoritmo de ordenação ☐
- Estratégias de Cache ☐
- Infraestrutura em Nuvem ☐
- Load Balancer ☐
- Tratamento de Exceções ☐
- Autenticação oAuth ☐
- Função de criptografia ☐
- Versão Mobile do Software ☐
- SOLID ☐
- Storage em Cloud ☐
- Comunicação síncrona ☐

Dinâmica sobre **A**rquitetura de Software vs **D**esign de Software.
Classifique os itens com **A** ou **D**

Sistema em microserviços		A
Uso de REST		A
Classe UsuarioService		D
Endpoint /api/login		D
Uso de mensageria		A
Sistema distribuído		A
Separação		A
frontend/backend		D
Método validar()		D
Classe MessageHandler		D
Uso de cache Redis		A
Classe CacheManager		D
API Gateway		A
Sistema tolerante a falhas		A
Camada de Repositório		A
Interface Repository		D
Comunicação assíncrona		A
Padrão de Projeto Adapter		D
Uso de PostgreSQL		A
Algoritmo de ordenação		D
Estratégias de Cache		A
Infraestrutura em Nuvem		A
Load Balancer		A
Tratamento de Exceções		D
Autenticação oAuth		A
Função de criptografia		D
Versão Mobile do Software		A
SOLID		D
Storage em Cloud		A
Comunicação síncrona		A

Então, quer dizer que, uma vez definida a Arquitetura de Software, o arquiteto pode “relaxar”, assumindo que todas as decisões serão corretamente aplicadas no Design do Software?

Resultado esperado pela Aplicação da Arquitetura

Resultado obtido pela Aplicação da Arquitetura

O trabalho de um Arquiteto de Software **não termina quando a Arquitetura é definida!**

O ARQUITETO DEVE:

- Comunicar a arquitetura
- Garantir sua correta aplicação
- Apoiar decisões de design
- Manter a integridade arquitetural

Definir a arquitetura é apenas o início. Garantir sua correta aplicação é um processo contínuo.

Conclusão

A **Arquitetura de Software** define a estrutura global do sistema, estabelecendo seus principais componentes, suas relações e as decisões fundamentais que impactam atributos de qualidade como **escalabilidade, desempenho, segurança e confiabilidade**. Ela representa o nível mais alto de organização do sistema e orienta sua construção.

O ***Design de Software***, por sua vez, define como esses componentes serão implementados, especificando **classes, interfaces, métodos e estruturas internas**. Ele detalha tecnicamente as decisões arquiteturais, transformando a arquitetura em uma solução implementável.

Arquitetura e Design são complementares e essenciais para o desenvolvimento de sistemas de software bem estruturados e sustentáveis.

Exercício Prático:

Proposta de Arquitetura de Software

Especificação do Sistema

A universidade deseja desenvolver um sistema chamado **PUC Connect**, com o objetivo de permitir a comunicação e interação entre alunos e professores.

Alunos:

- Visualizar disciplinas matriculadas
- Enviar mensagens aos professores
- Receber avisos das disciplinas
- Acessar materiais disponibilizados

Professores:

- Enviar avisos
- Responder mensagens
- Publicar materiais

O sistema será acessado via navegador e dispositivos móveis.

Requisitos Não Funcionais (Arquiteturalmente Significativos)

RNF01 — O sistema deve suportar até 10.000 usuários simultâneos.

RNF02 — O sistema deve estar disponível via navegador web e dispositivos móveis.

RNF03 — O sistema deve persistir todas as mensagens e avisos de forma permanente, garantindo sua disponibilidade para consulta futura.

RNF04 — O sistema deve permitir futura expansão.

RNF05 — O sistema deve apresentar tempo de resposta máximo de 3 segundos para 95% das requisições.

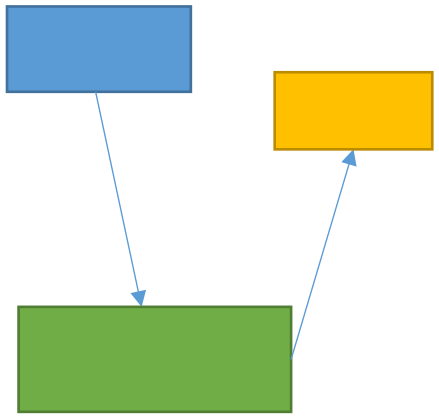
RNF06 — O sistema deve possuir disponibilidade mínima de 99%, permitindo no máximo 3,65 dias de indisponibilidade por ano.

Exercício Prático:

Proposta de Arquitetura de Software

Orientações:

- Abrir uma tarefa no Canvas para postagem das propostas de modelagem arquitetural (Professor)
- Os trabalhos devem ser postados até as 18:00 hs no dia da próxima aula, onde realizaremos a correção do desafio
- Apenas o líder do grupo deverá postar o diagrama em formato png ou jpg constando o nome completo de todos os integrantes
- Apresentar opção de ferramenta web (Professor)

PUC Connect
Diagrama Arquitetural

Grupo:
-Nome Completo
-Nome Completo
-Nome Completo
-Nome Completo